

HYDROROUTE-C

Overview & Architecture

This program is a C++ command-line application designed for emergency disaster response operations. It addresses two primary challenges during natural disasters (like floods):

Resource & Capacity Tracking: Managing shelter capacity and critical supplies (food and medical kits) using persistent binary file storage.

Safe Evacuation Routing: Finding the shortest safe path between geographical locations while filtering out impassable flooded roads using Dijkstra's algorithm.

Technical Component Breakdown

1. Data Structures & Constants C++

```
#define INF INT_MAX
#define MAX_NAME_LEN 50
#define FILE_NAME "relief_camps.dat"
```

```
struct ReliefCamp {
    int campID;
    char name[MAX_NAME_LEN];
    int maxCapacity;
    int currentOccupancy;
    double foodStockKg;
    double medicalKits;
};

struct RoadEdge {
    int toNode;
    int distanceKm;
    double floodWaterDepthMeters;
};
```

ReliefCamp Structure: Fixed-size data structure designed for binary file I/O. Using `char name[MAX_NAME_LEN]` instead of `std::string` guarantees a predictable, constant memory footprint per record (`sizeof(ReliefCamp)`), enabling binary stream reads and writes.

RoadEdge Structure: Represents a directed edge in a graph. In addition to distance (`distanceKm`), it tracks `floodWaterDepthMeters` to serve as a dynamic safety constraint.

2. Module 1: registerCamp() (Data Persistence)

Purpose: Collects relief camp details from the user and appends them to a binary file (`relief_camps.dat`).

Key Mechanics:

Uses `cin.ignore()` to flush residual newline characters left by numerical input before reading string data with `cin.getline()`.

Opens file mode `ios::binary | ios::app` to append records without overwriting existing data.

Writes memory blocks directly via `outFile.write(reinterpret_cast<char*>(&camp), sizeof(ReliefCamp));`.

3. Module 2: displayCamps() (Report Generation)

Purpose: Reads all camp records sequentially from the binary file and displays them in a tabular format.

Key Mechanics:

Opens file stream using `ios::binary`.

Reads through the file in a while loop using `inFile.read(...)`.

Formats output dynamically using standard I/O manipulators (`<iomanip>`) such as `left` and `setw()`.

4. Module 3: calculateEvacuationRoute() (Flood-Aware Pathfinding) Purpose: Computes the optimal safe route from source (<i>Guwahati Hub</i>) to target (<i>Majuli Shelter</i>) using Dijkstra's shortest-path algorithm. Key Mechanics:
Graph Construction: Represents locations as graph nodes (0 to 4) and roads as edges (vector<vector<RoadEdge>>).
Flood Safety Filter: During edge relaxation, any road with floodWaterDepthMeters > 0.5 is skipped:
C++ if (edge.floodWaterDepthMeters > 0.5) continue;
<i>Example:</i> Node 1 (<i>Morigaon</i>) to Node 2 (<i>Nagaon</i>) has a water depth of 0.8 m, so the algorithm bypasses it and takes the safe route via Node 0 → Node 2 (110 km).
Priority Queue Optimization: Uses priority_queue<pair<int, int>, ..., greater<...>> to extract the unvisited node with the smallest distance in $O(\log V)$ time.
Path Reconstruction: Maintains a parent[] array to backtrack and print the safe sequence of districts once the target is reached.
5. Module 4: checkResourceAlerts() (Resource Shortage Monitoring) Purpose: Scans binary records to identify camps facing critical shortages or overcrowding. Key Mechanics:
Checks two critical threshold conditions:
$\text{Food Stock} < 100 \text{ kg} \vee \text{Current Occupancy} \geq \text{Max Capacity}$
Emits prioritized warning logs for camps meeting either condition.
6. Main Control Loop (main())
Purpose: Provides a console menu interface with robust input validation.
Input Validation: Handles invalid non-numeric user inputs using cin.clear() (resets stream flags) and cin.ignore(1000, '\n') (discards bad buffer contents) to prevent infinite input loops.